LAPORAN PRAKTIKUM STRUKTUR DATA

“QUEUE”

Disusun Oleh:

Aryahiyahul Fikra

2511532026

Dosen Pengampu:

Dr. Wahyudi S.T, M.T

Asisten Pembimbing:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Daftar Isi

Daftar Isi....2
BAB I....3
PENDAHULUAN....3
1.1 Latar Belakang....3
1.2 Tujuan Praktikum....3
BAB II....4
PEMBAHASAN....4
2.1 Program Queue Menggunakan Array....4
2.2 Program Driver Queue Array....5
2.3 Program Queue Menggunakan LinkedList....6
2.4 Program Iterasi Queue....7
2.5 Program Reverse Queue....8
BAB III....9
PENUTUP....9
3.1 Kesimpulan....9
TUGAS QUEUE....10

BAB I
PENDAHULUAN

1.1 Latar Belakang

Queue merupakan salah satu struktur data linear yang menggunakan prinsip **FIFO (First In First Out)**, yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Struktur data ini banyak digunakan dalam berbagai sistem seperti antrian proses, manajemen data, dan simulasi sistem.

Dalam praktikum ini, dilakukan implementasi Queue menggunakan beberapa pendekatan, yaitu:

- Queue berbasis array
- Queue menggunakan LinkedList
- Iterasi Queue
- Reverse Queue menggunakan Stack

Dengan memahami berbagai implementasi ini, diharapkan mahasiswa dapat mengetahui cara kerja Queue secara lebih mendalam.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Queue
2. Mengimplementasikan Queue menggunakan array dan LinkedList
3. Mengetahui operasi dasar Queue (enqueue, dequeue, peek, dll)
4. Memahami cara membalik (reverse) Queue menggunakan Stack
5. Menganalisis hasil implementasi program

BAB II
PEMBAHASAN

2.1 Program Queue Menggunakan Array

Program ini mengimplementasikan Queue secara manual menggunakan array.

Kode program:

```java
package pekan4_2511532026;

public class QueueArray_2511532026 {
    int front_2026,rear_2026,size_2026;
    int capacity_2026;
    int array_2026[];

    public QueueArray_2511532026(int capacity) {
        this.capacity_2026 = capacity;
        front_2026=this.size_2026 =0;
        rear_2026=capacity - 1;
        array_2026=new int [this.capacity_2026];
    }

    boolean isFull_2026 (QueueArray_2511532026 queue) {
        return (queue.size_2026 == queue.capacity_2026);

    }

    boolean isEmpty_2026 (QueueArray_2511532026 queue) {
        return (queue.size_2026 == 0);
    }
    void enqueue_2026(int item) {
        if (isFull_2026(this))
            return;

        this.rear_2026 = (this.rear_2026 + 1) % this.capacity_2026;
        this.array_2026[this.rear_2026] = item;
        this.size_2026 = this.size_2026 + 1;

        System.out.println(item + " enqueued to queue");
    }

    int dequeue_2026() {
        if (isEmpty_2026(this))
            return Integer.MIN_VALUE;

        int item = this.array_2026[this.front_2026];
        this.front_2026 = (this.front_2026 + 1) % this.capacity_2026;
        this.size_2026 = this.size_2026 - 1;
        return item;
    }

    int front_2026() {
        if (isEmpty_2026(this))
            return Integer.MIN_VALUE;

        return this.array_2026[this.front_2026];
    }

    int rear_2026() {
        if (isEmpty_2026(this))
            return Integer.MIN_VALUE;

        return this.array_2026[this.rear_2026];
    }

    // mencetak elemen antrian
    void display_2026() {
        int i;

        if (front_2026 == rear_2026) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }

        // kunjungi dari belakang dan cetak
        for (i = front_2026; i < rear_2026; i++) {
            System.out.printf(" %d <-- ", array_2026[i]);
        }

        return;
    }
}
```

Penjelasan:

- Variabel:
    - front → posisi depan
    - rear → posisi belakang
    - size → jumlah elemen
    - capacity → kapasitas array
- Method utama:
    - enqueue() → menambah data

- dequeue() → menghapus data
- front() → melihat elemen depan
- rear() → melihat elemen belakang

2.2 Program Driver Queue Array

Program ini digunakan untuk menjalankan Queue array.

Kode program:

```java
package pekan4_2511532026;

public class QueueArrayDriver_2511532026 {
    public static void main(String[] args) {
        QueueArray_2511532026 queue_2026 = new QueueArray_2511532026(1000);

        queue_2026.enqueue_2026(10);
        queue_2026.enqueue_2026(20);
        queue_2026.enqueue_2026(30);
        queue_2026.enqueue_2026(40);
        System.out.println("Item di depan " + queue_2026.front_2026());
        System.out.println("Item paling belakang " + queue_2026.rear_2026());
        System.out.println("tampilan queue");
        queue_2026.display_2026();
        System.out.println();
        System.out.println(queue_2026.dequeue_2026() + " dihapus dari queue");
        System.out.println("item di depan: " + queue_2026.front_2026());
        System.out.println("item dibelakang: " + queue_2026.rear_2026());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue_2026.display_2026();
    }
}
```

Penjelasan:

- Menambahkan data: 10, 20, 30, 40
- Menampilkan:
    - Elemen depan
    - Elemen belakang
- Menghapus satu data (dequeue)

Hasil:

- Data pertama yang masuk (10) akan keluar lebih dulu → sesuai FIFO

2.3 Program Queue Menggunakan LinkedList

Program ini menggunakan struktur bawaan Java (LinkedList) sebagai Queue.

Kode program:

```java
package pekan4_2511532026;

import java.util.*;

public class QueueLinkedList_251153026 {

    public static void main(String[] args) {
        Queue<Integer> q_2026 = new LinkedList<>();
        for ( int i=0; i<6; i++)
            q_2026.add(i);

        System.out.println("elemen antrian " +q_2026);

        int hapus = q_2026.remove();
        System.out.println("hapus elemen =" +hapus);
        System.out.println(q_2026);

        int depan = q_2026.peek();
        System.out.println("kepala antrian = " + depan);

        int banyak = q_2026.size();
        System.out.println("size antrian = " + banyak);

    }

```

Penjelasan:

- add() → menambah data
- remove() → menghapus data
- peek() → melihat elemen depan tanpa menghapus
- size() → jumlah elemen

Kelebihan:

- Tidak perlu menentukan kapasitas
- Lebih fleksibel dibanding array

2.4 Program Iterasi Queue

Program ini menampilkan isi Queue menggunakan iterator.

Kode program:

```java
package pekan4_2511532026;
import java.util.*;
public class IterasiQueue_2511532026 {

    public static void main(String args[]) {

            Queue<String> q_2026 = new LinkedList<>();

            q_2026.add("Praktikum");
            q_2026.add("Struktur");
            q_2026.add("Data");
            q_2026.add("Dan");
            q_2026.add("Algoritma");

            Iterator<String> iterator = q_2026.iterator();

            while (iterator.hasNext()) {
                System.out.print(iterator.next() + " ");
            }
        }
    }
```

Penjelasan:

- Menggunakan Iterator
- Loop:

  while (iterator.hasNext())

- Menampilkan data satu per satu

Fungsi:

Digunakan untuk membaca isi Queue tanpa mengubah datanya.

2.5 Program Reverse Queue

Program ini membalik isi Queue menggunakan Stack.

Kode program:

```java
package pekan4_2511532026;
import java.util.LinkedList;

public class ReserveData_2511532026 {
    public static void main(String[] args) {
        Queue<Integer> q_2026 = new LinkedList<Integer>();
        q_2026.add(1);
        q_2026.add(2);
        q_2026.add(3);
        System.out.println("sebelum reverse" + q_2026);
        Stack<Integer> s_2026 = new Stack<Integer>();
        while (!q_2026.isEmpty()) {
            s_2026.push(q_2026.remove());
        }
        while (!s_2026.isEmpty()) {
            q_2026.add(s_2026.pop());
        }
        System.out.println("sesudah reverse : " +q_2026);

    }

}
```

Penjelasan:

Langkah-langkah:

1. Semua elemen Queue dimasukkan ke Stack
2. Stack membalik urutan (LIFO)
3. Data dikembalikan ke Queue

Konsep:

- Queue → FIFO
- Stack → LIFO

kombinasi antara FIFO dan LIFO ini menghasilkan reverse data.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue bekerja dengan prinsip FIFO (First In First Out). Implementasi Queue dapat dilakukan menggunakan array maupun LinkedList, dimana masing-masing memiliki kelebihan dan kekurangan. Operasi dasar seperti enqueue, dequeue, peek, dan size sangat penting dalam pengelolaan data Queue. Selain itu, Queue juga dapat dimodifikasi, seperti melakukan reverse menggunakan bantuan Stack. Dengan demikian, pemahaman Queue sangat penting dalam pengembangan program yang melibatkan sistem antrian.

TUGAS QUEUE

Kode Program:

```java
package pekan4_2511532026;
import java.util.Scanner;

class Queue_2511532026 {
    String[] queue_2026;
    int front_2026, rear_2026, size_2026, max_2026;

    public Queue_2511532026(int max_2026) {
        this.max_2026 = max_2026;
        this.front_2026 = 0;
        this.size_2026 = 0;
        this.rear_2026 = max_2026 - 1;
        this.queue_2026 = new String[max_2026];
    }

    public boolean isEmpty_2026() {
        return (size_2026 == 0);
    }

    public boolean isFull_2026() {
        return (size_2026 == max_2026);
    }

    public void enqueue_2026(String data_2026) {
        if (isFull_2026()) {
            System.out.println("Antrian penuh!");
            return;
        }

        rear_2026 = (rear_2026 + 1) % max_2026;
        queue_2026[rear_2026] = data_2026;
        size_2026++;

        System.out.println("Data berhasil ditambahkan ke antrian");
    }

    public void dequeue_2026() {
        if (isEmpty_2026()) {
            System.out.println("Antrian kosong!");
            return;
        }

        String data_2026 = queue_2026[front_2026];
        front_2026 = (front_2026 + 1) % max_2026;
        size_2026--;
        System.out.println(data_2026 + " telah dilayani");
    }

    public void display_2026() {
        if (isEmpty_2026()) {
            System.out.println("Antrian kosong!");
            return;
        }

        System.out.println("Isi antrian:");
        for (int i_2026 = 0; i_2026 < size_2026; i_2026++) {
            System.out.println((i_2026 + 1) + ". " + queue_2026[(front_2026 + i_2026) % max_2026]);
        }
    }

    public void Reverse_2026() {
        if (isEmpty_2026()) {
            System.out.println("Antrian kosong!");
            return;
        }

        String[] temp_2026 = new String[size_2026];

        for (int i_2026 = 0; i_2026 < size_2026; i_2026++) {
            temp_2026[i_2026] = queue_2026[(front_2026 + i_2026) % max_2026];
        }

        for (int i_2026 = 0; i_2026 < size_2026; i_2026++) {
            queue_2026[(front_2026 + i_2026) % max_2026] = temp_2026[size_2026 - 1 - i_2026];
        }

        display_2026();
    }
}

public class AntrianLoket_2511532026 {
    public static void main(String[] args) {
        Scanner input_2026 = new Scanner(System.in);
        Queue_2511532026 antrian_2026 = new Queue_2511532026(10);
        int pilihan_2026;
        String nama_2026;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan_2026 = input_2026.nextInt();
            input_2026.nextLine();

            switch (pilihan_2026) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    nama_2026 = input_2026.nextLine();
                    antrian_2026.enqueue_2026(nama_2026);
                    break;

                case 2:
                    antrian_2026.dequeue_2026();
                    break;

                case 3:
                    antrian_2026.display_2026();
                    break;

                case 4:
                    antrian_2026.Reverse_2026();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan_2026 != 5);

        input_2026.close();
    }
}
```

Penjelasan Program AntrianLoket_2511532026

1. Deskripsi Program

Program ini merupakan implementasi struktur data **Queue (antrian)** menggunakan array dalam bahasa Java. Queue bekerja dengan prinsip **FIFO (First In First Out)**, yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Pada program ini, queue digunakan untuk mensimulasikan antrian pelanggan di loket pelayanan. Pelanggan yang datang akan ditambahkan ke dalam antrian, kemudian dilayani sesuai urutan kedatangannya.

2. Struktur Program

Program terdiri dari dua class dalam satu file, yaitu:

a. Class Queue_2511532026

Class ini berfungsi untuk mengelola seluruh operasi antrian, seperti menambah, menghapus, menampilkan, dan membalik isi antrian.

b. Class AntrianLoket_2511532026

Class ini berisi method main() yang berfungsi sebagai pengendali utama program serta menampilkan menu interaktif kepada pengguna.

3. Atribut pada Class Queue

- queue_2026[] : array untuk menyimpan data nama pelanggan.
- front_2026 : menunjukkan posisi elemen paling depan pada antrian.
- rear_2026 : menunjukkan posisi elemen paling belakang pada antrian.
- size_2026 : menyimpan jumlah data yang sedang berada dalam antrian.
- max_2026 : kapasitas maksimum antrian.

4. Constructor

Constructor Queue_2511532026(int max_2026) digunakan untuk menginisialisasi queue.

- max_2026 diisi sesuai kapasitas yang diinginkan.
- front_2026 diatur ke 0.
- size_2026 diatur ke 0, menandakan antrian masih kosong.
- rear_2026 diatur ke max_2026 - 1.
- Array queue_2026 dibuat sesuai kapasitas.

5. Method pada Class Queue

a. isEmpty_2026()

Method ini digunakan untuk memeriksa apakah antrian kosong.

- Mengembalikan nilai true jika size_2026 == 0.
- Mengembalikan nilai false jika antrian berisi data.

b. isFull_2026()

Method ini digunakan untuk memeriksa apakah antrian penuh.

- Mengembalikan nilai true jika size_2026 == max_2026.
- Mengembalikan nilai false jika masih ada ruang kosong.

c. enqueue_2026(String data)

Method ini berfungsi untuk menambahkan pelanggan ke belakang antrian.

Langkah kerja:

1. Memeriksa apakah antrian penuh.
2. Jika penuh, program menampilkan pesan "Antrian penuh!".
3. Jika tidak penuh, posisi rear_2026 digeser satu langkah.
4. Data pelanggan disimpan pada posisi tersebut.
5. size_2026 bertambah satu.

d. dequeue_2026()

Method ini berfungsi untuk menghapus pelanggan dari depan antrian.

Langkah kerja:

1. Memeriksa apakah antrian kosong.
2. Jika kosong, program menampilkan pesan "Antrian kosong!".
3. Jika tidak kosong, data pada posisi front_2026 diambil.
4. Posisi front_2026 digeser satu langkah.
5. size_2026 dikurangi satu.
6. Nama pelanggan yang dilayani ditampilkan.

e. display_2026()

Method ini digunakan untuk menampilkan seluruh isi antrian.

- Jika antrian kosong, tampil pesan "Antrian kosong!".

- Jika berisi data, seluruh elemen ditampilkan mulai dari depan hingga belakang sesuai urutan antrian.

f. Reverse_2026()

Method ini digunakan untuk membalik urutan seluruh isi antrian.

- Data paling depan menjadi paling belakang.
- Data paling belakang menjadi paling depan.
- Setelah dibalik, isi antrian ditampilkan kembali.

6. Alur Program Utama

Pada method main(), program akan:

1. Membuat objek queue dengan kapasitas tertentu.
2. Menampilkan menu pilihan.
3. Membaca input pengguna.
4. Menjalankan operasi sesuai pilihan menu.
5. Mengulangi proses hingga pengguna memilih menu keluar.

7. Menu Program

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

8. Konsep FIFO pada Program

Program menerapkan prinsip FIFO:

- Pelanggan yang pertama masuk akan dilayani terlebih dahulu.
- Pelanggan yang datang belakangan harus menunggu giliran.

Contoh:

- Andi masuk pertama.
- Budi masuk kedua.
- Saat dequeue dilakukan, Andi akan dilayani terlebih dahulu.

9. Kesimpulan

Program ini berhasil mengimplementasikan struktur data Queue menggunakan array dalam bahasa Java. Program mampu melakukan operasi dasar queue seperti enqueue, dequeue, display, pengecekan kondisi penuh dan kosong, serta reverse. Implementasi ini sangat sesuai untuk simulasi antrian pelanggan pada loket pelayanan karena menerapkan prinsip FIFO secara efektif.